rtacc プロジェクト

社内中間報告

LLVM 技術の活用と今後の可能性

アジェンダ

1. LLVM 技術とその活用 — 様々な可能性
2. 中間報告の位置づけと、研究時間について
3. 今後の研究予定と成果イメージ
4. Visual Studio Community と rtacc.exe の位置づけ
5. rtacc.exe の多言語対応とコマンド例

1. LLVM 技術とその活用

様々な可能性が見えてきたこと

- **LLVM / Clang** を基盤にしたツールチェーンにより、INtime RTA 向けビルドを **Visual Studio に依存せず** 実現
- **IL（Instruction List）→ LLVM IR**（llil）、**ST（Structured Text）→ LLVM IR**（llst）により、PLC 系言語を共通中間表現に集約
- 一貫した **LLVM IR** パイプラインにより、
  - 最適化の共通化
  - x86 ネイティブコードへのコンパイル実行
  - 将来的なデバッグ・解析ツール（例: lldb.rta）の接続が現実的に

2. 中間報告の位置づけと、研究時間について

- **もともと** PLCplus の将来改造について LLVM を検討していた
- 検討を進めるうちに、**その枠に収まらない可能性**（性能大幅向上、デバッグ実行、脱VS依存など）が見えてきた
- 本報告は、その**可能性が出てきたことを共有する**中間報告です
- 可能性をさらに裏付けるため、**もう少し研究する時間**をいただきたいと考えています

3. 今後の研究予定と成果イメージ

項目	内容
処理性能比較	PLCplus 製品の **処理性能向上の比較**（従来 IL インタプリタ vs コンパイル実行）
デバッグ・実行	仮想マイコン製品の **デバッグ実行機能の実現可能性**（lldb.rta の開発検討）
IL→LL による高速化	LLVM IR を使うことで、PLCplus 製品（**IL→LL 変換**）の **高速化** を検証
コンパイル実行	PLCplus が **x86 ネイティブコードを実行できる**こと（コンパイル実行）の確認
最適化の最大化	**IEC ライブラリ / RT-Edge** も LL 化する試みにより、最適化を最大に

成果イメージ（1）

PLCplus 処理性能の向上

- **従来**: IL をインタプリタで逐次実行 → オーバーヘッドが大きい
- **本方式**: IL → LLVM IR → ネイティブコード（コンパイル実行）… AI で試作した（llil）
- **これまでの成果のひとつ**: **ST 言語 → LL → PLCplus から実行**（ネイティブ rta）も実現済み（llst）
- **期待**: 従来の IL インタプリタ実行と比べて **数十倍〜数百倍** の処理性能向上の可能性が高い
- 引き続きの研究で、**定量的な比較データ** を取得し、報告する予定です

成果イメージ（2）

仮想マイコン・デバッグ実行（lldb.rta）

- **目的**: 仮想マイコン製品で **デバッグ・実行機能** を実現する可能性の検討
- **手段**: LLVM エコシステムのデバッガ（LLDB）を INtime RTA 向けに応用した **lldb.rta** の開発を試みる
- **成果**: 実現可能性の評価と、可能であればプロトタイプの提示

成果イメージ（3）

LLVM IR による PLCplus 高速化

- **IL → LL 変換**（llil）により、PLCplus の IL プログラムを **LLVM IR** に変換
- 同一パイプラインで **最適化** と **x86 ネイティブコード生成** を実施
- **コンパイル実行** により、PLCplus が **x86 ネイティブコードを実行** できることを確認
- 一貫して LLVM IR を使うことで、最適化の効果を最大限に引き出せる

成果イメージ（4）

IEC ライブラリ / RT-Edge の LL 化

- **IEC ライブラリ** や **RT-Edge** も LLVM IR（.ll）に寄せる試みを行うことで、
  - 全体が一つの IR に統一され、
  - **最適化が最大** になる（インライン化・定数伝播・デッドコード削除などが横断的に効く）
- 引き続きの研究で、**どこまで LL 化を試みるか** と **効果** を整理して報告します

4. Visual Studio Community と rtacc.exe

課題	rtacc.exe の役割
Visual Studio Community は **商用利用に制限** がある	rtacc.exe をビルドドライバとして用い、**LLVM/Clang** でコンパイル・リンクすることで、**VS に依存しない** 商用利用可能なビルド環境を構築
ライセンスや環境の統一が難しい	オープンソースの Go/Rust で実装された rtacc / llil / llst により、**再現可能でライセンス的にクリーン** なツールチェーンを提供

→ **rtacc.exe が Visual Studio Community の商用利用制限をカバー** する形で利用できます。

5. rtacc.exe のいろいろな言語対応

入力	役割	変換・ビルド
.c	C ソース	clang で .ll → .asm → .obj（LLVM）または cl で .obj（VS）
.st	Structured Text	**llst.exe** で .ll → clang で .asm → .obj
.il	Instruction List	**llil.exe** で .ll → clang で .asm → .obj
.obj / .o / .lib	既存オブジェクト・ライブラリ	そのままリンクに使用

→ **C / ST / IL** を同一プロジェクトで混在させ、一括して **.rta** にビルドできます。

rtacc のコマンド例（1）

基本

```
# 単一 C ファイル → app.rta
rtacc main.c -o app.rta

# 複数ソース + オブジェクト + ライブラリ
rtacc main.c util.c foo.obj -o app.rta --libs "a.lib;b.lib"

# コンパイルのみ（リンクしない）
rtacc main.c -c -o app.rta
```

```
# JSON プロジェクトでビルド（ターゲット指定可）
rtacc project.json
rtacc project.json -target default --toolset llvm
```

rtacc のコマンド例（2）

オプション

```
# ツールセット: LLVM または VS2017
rtacc project.json --toolset llvm
rtacc main.c -o app.rta --toolset vs2017

# 最適化レベル
rtacc main.c -o app.rta -O 2

# インクルード・定義・追加 lib
rtacc main.c -o app.rta -I "include;other" -D "FOO=1" --libs "mylib.lib"

# 出力ディレクトリ
rtacc project.json -output-dir out
```

rtacc のコマンド例（3）

多言語（C + ST + IL）の例

```
# project.json の sources に main.c, logic.st, seq.il を並べて一括ビルド
rtacc project.json -o out/app.rta
```

- **llst** が .st → .ll、**llil** が .il → .ll を担当
- rtacc が .c / .ll を clang で .asm → .obj にし、最後にリンクして .rta を生成
- **PATH** または **RTACC_BIN** に rtacc / llil / llst を置いておく必要があります

まとめ

- **LLVM 技術の活用** により、INtime RTA 向けビルドの脱 VS 依存、IL/ST の LL 化、コンパイル実行による **大幅な性能向上の可能性** が見えてきました。
- もう少し研究を続けさせていただくことで、**PLCplus 処理性能比較**、**lldb.rta の実現可能性**、**IEC/RT-Edge の LL 化** などを整理し、成果を報告する予定です。
- **rtacc.exe** は、VS Community の商用利用制限をカバーし、**C / ST / IL** に対応したビルドドライバとして、様々なコマンド例で利用できます。

ご清聴ありがとうございました。